yafo – Architektur & Features

Nov 2, 2026 · @Jon

Überblick

yafo („yet another family organizer“) ist ein Familien-Organizer für iPhone und iPad: geteilte und eigene Kalender, Einkaufsliste, Todos und Erinnerungen. Alle Daten liegen in iCloud, ein eigener Server ist nicht nötig.

- **Zielgruppe:** Familien, in denen alle Erwachsenen Apple-Geräte nutzen. Kinder ohne Apple Account sind Profile; Teenager mit eigenem Account werden Mitglied.
- **Privat:** Daten nur in der iCloud der Familie, KI-Funktionen laufen auf dem Gerät.
- **Kein Server:** keine Betriebskosten, keine eigene Nutzerverwaltung. Eingeladen wird über den Apple Account.
- **Übersichtlich:** klare, schlichte Oberfläche ohne Schnörkel, Fokus auf die Monatsübersicht.
- **Offen:** Export und Backup in Standardformaten, damit niemand in der App gefangen ist.
- **Veröffentlichung:** kostenlos und weltweit im App Store, ohne In-App-Käufe, über einen privaten Developer Account (DSA: kein Händler). Oberfläche auf Deutsch und Englisch, Quellcode öffentlich auf GitHub.
- **Plattformen:** iOS und iPadOS ab Version 27, auf Apple-Silicon-Macs als iPad-App. Keine Apple Watch in V1.

Feature-Umfang Version 1

Version 1 umfasst fünfzehn Features. Ein eigener CalDAV-Client sowie Android, Web, Apple Watch und eine Wochenansicht sind bewusst nicht enthalten.

Feature	Was es kann	Technische Umsetzung
Geteilte Kalender	Gemeinsame Termine, für alle Mitglieder sichtbar; mehrere geteilte Kalender möglich	Geteilte CloudKit-Zone pro Familie
Mehrere Familien	Ein Nutzer kann Mitglied mehrerer Familien sein, z. B. mit den Großeltern	Ein CKShare pro Familie
Eigener Kalender	Termine, die nur man selbst sieht	Private CloudKit-Datenbank

Feature	Was es kann	Technische Umsetzung
Kinderprofile und Teilnehmer	Kinder ohne Apple Account als Profil; pro Termin wählbar, wer dabei ist	Member-Profile, Teilnehmer-Beziehung am Termin
Einkaufsliste	Pro Familie und privat („Nur ich“); Abhaken wird synchronisiert, „Zuletzt gekauft“ und Autovervollständigung	Ein Datensatz pro Artikel
Todos	Pro Familie und privat; Fälligkeit, zuständige Person (auch Kinder), Wiederholung	Ein Datensatz pro Aufgabe
Erinnerungen	Benachrichtigung bei fälligen Terminen und Aufgaben, an die Teilnehmer; Zeit pro Person anpassbar	Lokale Notifications
Kommentare am Termin	Chat-Verlauf in der Beschreibung, klar erkennbar, wer was geschrieben hat; eigene Kommentare bearbeiten und löschen	Ein Datensatz pro Kommentar
Kalender-Abos	Feiertage und Schulferien für DE, AT und CH pro Region zur Auswahl, eigene ICS-Adressen; festgelegt pro Kalender	OpenHolidays-Daten aus eigenem Spiegel, ICS-Parser
Import	Einmalige Migration, z. B. vom Google Kalender	EventKit, ICS-Adresse oder .ics-Datei
Externe Kalender	Geräte-Kalender (Arbeit, Google, CalDAV) in der eigenen Ansicht einblenden	EventKit, nur auf dem Gerät
Export und Backup	Daten als ICS, CSV oder vollständiges Backup exportieren und wiederherstellen	Datei-Export, EventKit, JSON
Screenshot zu Termin	Screenshot teilen, Termin wird erkannt und vorgeschlagen	Share Extension und Foundation Models
Widgets	Heute und morgen, nächste Termine	WidgetKit

Feature	Was es kann	Technische Umsetzung
Jahresansicht	Zwölf Monate auf einen Blick, mit Ferien und Feiertagen	SwiftUI

UI-Konzept

Die Monatsansicht ist der Startbildschirm. Die Farbe zeigt den Kalender, nicht die Person. Jeder Kalender startet mit der Standardfarbe seines Typs, jeder Nutzer kann sie für sich ändern.

Farbsystem

Kalendertyp	Standardfarbe	Darstellung im Monatsraster
Geteilter Kalender	Blau, weitere geteilte Kalender in eigenen Farben	Farbiger Balken bzw. Punkt
Eigener Kalender	Grün	Farbiger Balken bzw. Punkt
Feiertage	Rot	Tageszahl rot, Name in der Tagesliste
Schulferien	Helles Orange	Hintergrundfläche über die Ferientage
Externe Kalender (EventKit)	Grau	Dezenter Punkt

Ferien als Fläche statt als Punkt, damit sie keine Termine aus der Zelle verdrängen. Wer bei einem Termin dabei ist, zeigen Initialen-Avatare in der Tagesliste, z. B. Zoo mit Vater, Mutter und Tochter. Ein Termin ohne Teilnehmer gilt für die ganze Familie.

Monatsansicht

- Raster mit höchstens drei Markierungen pro Tag, darüber hinaus „+2“.
- Tippen auf einen Tag zeigt darunter die Tagesliste.
- Wischen wechselt den Monat, „Heute“ springt zurück, der heutige Tag ist hervorgehoben.
- Filter nach Person: nur die Termine, bei denen z. B. die Tochter dabei ist.
- Kalender einzeln ein- und ausblendbar.

Jahresansicht

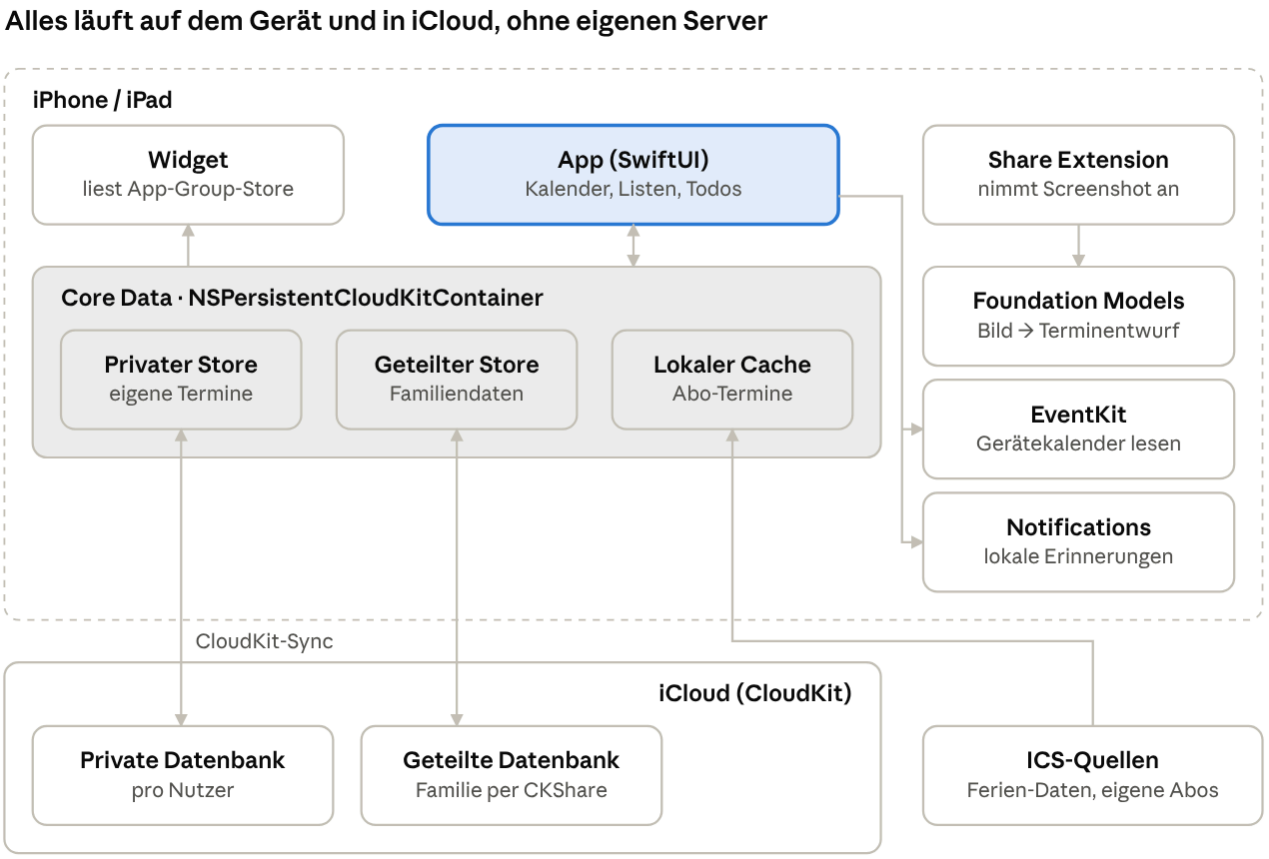
- Zwölf Mini-Monate: 3 × 4 auf dem iPhone, 4 × 3 auf dem iPad.
- Ferienflächen und Feiertage sichtbar, Tage mit Terminen dezent markiert.
- Tippen auf einen Monat öffnet die Monatsansicht.

Weitere Bildschirme

- **Navigation:** Tab Bar mit Kalender, Einkauf, Todos und Familie. Bei mehreren Familien ein Umschalter oben in Einkauf und Todos; dort steht auch „Nur ich“ für die privaten Listen. Neues von anderen (Termine, Kommentare, Artikel) zeigt ein „Neu“-Marker, keine Push-Nachricht.
- **iPad:** Monat links, Tagesliste rechts nebeneinander.
- **Termin anlegen:** Kalender wählen, dann Teilnehmer per Chips antippen (Erwachsene und Kinderprofile der Familie).
- **Termin-Detail:** Titel, Zeit, Kalender, Teilnehmer, darunter die Kommentare als Chat-Blasen mit Avatar, Name und Uhrzeit sowie ein Eingabefeld.
- **Familie:** Mitglieder einladen, Kinderprofile anlegen, geteilte Kalender verwalten, Farben ändern. Ohne Familie läuft die App mit eigenem Kalender und privaten Listen.

Architektur

CloudKit ist das Backend. Synchronisiert wird über NSPersistentCloudKitContainer mit zwei CloudKit-Stores und einem lokalen Cache. SwiftData scheidet aus, weil es kein CloudKit-Sharing unterstützt.



Architektur · App, Extensions, Stores und iCloud

Die App synchronisiert privaten und geteilten Store mit iCloud. Widget und Share Extension greifen über die App Group auf dieselben Daten zu, der Abo-Cache bleibt lokal.

Einladen in die Familie

1. Das erste Mitglied legt die Familie an und wird Owner. Das Family-Objekt liegt in seinem privaten Store.
2. `share(_:to:)` erzeugt eine eigene Zone mit einem `CKShare`. Alles, was an Family hängt, wird mitgeteilt.
3. Die Einladung geht über den System-Freigabedialog (`UICloudSharingController` bzw. `ShareLink`), z. B. per Nachrichten.
4. Das Mitglied tippt auf den Link, die App nimmt die Einladung an (`acceptShareInvitations`), und die Familiendaten erscheinen in seinem geteilten Store.
5. Neue Familienobjekte weist die App per `assign(_:to:)` dem richtigen Store zu: beim Owner dem privaten, bei Mitgliedern dem geteilten.

Alle Erwachsenen bekommen Schreibrechte; eine Rechte-Auswahl gibt es in V1 nicht. Wer nur mitlesen soll, bekommt eine eigene Familie. Einladen kann nur der Owner.

Jede Familie ist ein eigener `CKShare`. Ein Nutzer kann Owner der einen und Mitglied einer anderen Familie sein; die App zeigt die Kalender aller Familien gemeinsam an. Kinderprofile legt ein Erwachsener in der Familie an, sie haben keinen eigenen Zugang.

- **Termine bearbeiten:** Alle Mitglieder dürfen alle Termine ändern und löschen. Der Ersteller wird angezeigt, vor dem Löschen eines fremden Termins fragt die App nach.
- **Teenager mit Account:** werden normales Mitglied mit kind = Kind. Ein bestehendes Kinderprofil lässt sich mit dem Account verknüpfen, die bisherigen Teilnahmen bleiben.
- **Kind in zwei Familien:** pro Familie ein eigenes Profil, ohne Verknüpfung.
- **Austritt:** Termine, Kommentare und Todos bleiben, der Name erhält den Zusatz „(ehemalig)“. Auf dem Gerät der Person verschwinden die Familiendaten.

Owner übertragen

CloudKit selbst kann die Owner-Rolle nicht übertragen, weil die Familiendaten in der iCloud des Owners liegen. Die App löst das mit einem Umzug in einen neuen Share beim neuen Owner.

1. Der Owner wählt ein erwachsenes Mitglied als neuen Owner. Die App legt dafür einen OwnerTransfer-Datensatz in der Familie an.
2. Der neue Owner bestätigt. Seine App kopiert alle Familiendaten samt ihrer ids in seinen privaten Store und teilt sie als neuen Share.
3. Seine App fügt alle Erwachsenen über ihre cloudKitUserID als Teilnehmer hinzu, auch den bisherigen Owner, und schreibt den Link des neuen Shares in den OwnerTransfer-Datensatz.
4. Die Apps der Mitglieder lesen den Link, nehmen den neuen Share an und blenden die alte Familie aus.
5. Sobald alle umgezogen sind, löscht die App des bisherigen Owners den alten Share.

Während des Umzugs ist die Familie schreibgeschützt, damit keine Änderung verloren geht. Weil alle Verweise über die eigene id laufen, bleiben Kommentare, Teilnehmer und Autoren erhalten. Wer länger offline ist, zieht beim nächsten Öffnen nach; der alte Share bleibt dafür einige Tage bestehen.

Die Owner-Übertragung kommt in V1.1. Schon in Phase 1 prüft ein Prototyp, ob Mitglieder den neuen Share ohne Tippen annehmen. Ist der Owner ganz weg, stellt ein Mitglied aus seinem JSON-Backup eine neue Familie her und wird deren Owner; die anderen lädt es neu ein.

Datenmodell

Ein gemeinsames Core-Data-Modell für alle Stores. Der Store, in dem ein Objekt liegt, entscheidet, wer es sieht. Jede Familie ist ein eigener Share mit eigenen Kalendern, Listen und Mitgliedern.

Entität	Store	Wichtige Felder	Hinweis
Family	Privat (Owner), geteilt (Mitglieder)	id, name	Wurzel eines Shares; ein Nutzer kann mehrere haben
Member	Wie die Familie	id, displayName, initials, kind (Erwachsener oder Kind), cloudKitUserID, leftAt	Kinderprofile ohne cloudKitUserID, Teenager bekommen sie beim Verknüpfen; leftAt markiert ehemalige Mitglieder; die ID der Erwachsenen braucht die Owner-Übertragung
Calendar	Privat oder wie die Familie	id, name, kind (geteilt oder eigen), defaultColor	Mehrere geteilte Kalender pro Familie möglich
Event	Wie sein Kalender	id, title, start, end, isAllDay, timeZone, location, notes, recurrenceRule (RRULE), createdBy, importUID, defaultAlertOffset	Gehört zu genau einem Kalender
Event ↔ Member	Wie der Termin	Viele-zu-viele-Beziehung participants	Wer dabei ist; leer bedeutet ganze Familie
EventException	Wie der Termin	originalDate, isCancelled, geänderte Felder	Verschobene oder gelöschte Einzeltermine einer Serie
Comment	Wie der Termin	text, createdAt, editedAt, author	Ein Datensatz pro Nachricht

Entität	Store	Wichtige Felder	Hinweis
ShoppingItem	Wie die Familie oder privat	name, quantity, isChecked, checkedAt, sortKey	Ein Datensatz pro Artikel; abgehakte Artikel liefern „Zuletzt gekauft“ und die Autovervollständigung
TodoItem	Wie die Familie oder privat	title, dueDate, isDone, createdBy, assignee, recurrenceRule, defaultAlertOffset	Ein Datensatz pro Aufgabe; eine Serie rollt beim Abhaken auf das nächste Fälligkeitsdatum weiter
Subscription	Wie sein Kalender	calendar, sourceKey, url, kind, region	Abo eines Kalenders; beim geteilten Kalender vom Owner festgelegt
SubscribedEvent	Lokal, ohne CloudKit	sourceKey, Felder aus der ICS-Datei	Cache pro Quelle, nicht pro Kalender; wird bei jedem Abruf neu aufgebaut
CalendarAppearance	Privat	calendarID, color, isVisible	Eigene Farbe und Sichtbarkeit pro Kalender, nur für einen selbst
AlertPreference	Privat	itemID, alertOffset, isMuted	Eigene Erinnerungszeit oder Stummschalten pro Termin bzw. Todo; überschreibt die Vorgabe
ReadState	Privat	familyID, lastSeenAt	Grundlage für „Neu“-Marker und Badges, z. B. ungelesene Kommentare
OwnerTransfer	Wie die Familie	newOwnerID, status, newShareURL, startedAt	Steuert die Übertragung der Owner-Rolle

Regeln für das CloudKit-Schema

- Alle Beziehungen optional, keine geordneten Beziehungen, keine Unique Constraints.
- Attribute optional oder mit Standardwert.
- Nach dem ersten Produktiv-Deployment nur noch erweitern, nie umbenennen oder löschen.

- Keine Beziehungen zwischen Stores: Kommentare und Teilnehmer liegen immer im Store ihres Termins.
- Verweise über Stores hinweg (z. B. CalendarAppearance auf einen geteilten Kalender) nur über die gespeicherte id als Text.
- Jedes Objekt bekommt eine eigene UUID im Feld id, damit Backups eindeutig wiederhergestellt werden können.
- Reihenfolge der Einkaufsliste über ein eigenes Feld sortKey, nicht über ein Array.
- Importierte Termine behalten ihre `UID` in importUID, damit ein erneuter Import keine Duplikate erzeugt.

Kalender-Integration

Alle Abos und Importe laufen über einen gemeinsamen ICS-Parser. Nur EventKit braucht eigenen Code.

Vorgeschlagene Abos

- **Feiertage:** aus der OpenHolidays API für DE, AT und CH, pro Bundesland bzw. Kanton. Ein Datenstand liegt in der App, damit Feiertage auch beim ersten Start ohne Netz da sind.
- **Schulferien:** ebenfalls aus OpenHolidays (Daten unter ODbL-1.0): alle 16 deutschen Länder bis 2030, dazu AT und CH. Eine GitHub Action spiegelt die Daten wöchentlich in ein eigenes Repo, die App lädt sie von dort höchstens einmal pro Woche. In der App steht „Daten: OpenHolidays API, ODbL“. Verworfen wurde ferien-api-data: keine Lizenz, die meisten Länder enden 2025.
- **Auswahl:** im Onboarding „Familie anlegen“, „Einladung annehmen“ oder „später“, dann die Region (Bundesland oder Kanton) wählen; Feiertage und Ferien sind dann vorausgewählt und jederzeit abschaltbar. Gilt für den eigenen Kalender; beim Anlegen einer Familie fragt die App den Owner dasselbe für den geteilten Kalender.
- **Eigene Abos:** beliebige ICS- oder webcal-Adresse. Aktualisierung beim Öffnen der App, zusätzlich per Background Task ohne Garantie.

Abos pro Kalender

- Jeder Kalender hat eigene Abos. Beim geteilten Kalender legt der Owner sie in den Einstellungen fest, beim eigenen Kalender der Nutzer selbst.
- Die Abo-Einstellung liegt im Store des Kalenders und synchronisiert so an alle Mitglieder. Die Termine selbst lädt jedes Gerät von der Quelle und speichert sie nur lokal.

- CloudKit vergibt Schreibrechte pro Teilnehmer, nicht pro Datensatz. „Nur der Owner ändert Abos“ ist deshalb eine Regel in der App, keine Sperre in iCloud.

Doppelte Abos

Abonnieren Familienkalender und eigener Kalender dieselben Schulferien, erscheinen sie trotzdem nur einmal.

1. Jede Quelle hat einen eindeutigen Schlüssel, z. B. `ferien:BY`, `feiertage:BY` oder die normalisierte URL.
2. Der lokale Cache speichert Termine pro Quelle, nicht pro Kalender. Eine Quelle wird einmal geladen, egal wie viele Kalender sie abonnieren.
3. Die Ansicht zeigt jede Quelle einmal, sobald mindestens ein sichtbarer Kalender sie abonniert.
4. Beim Abonnieren im eigenen Kalender zeigt die App den Hinweis „Bereits über den Familienkalender abonniert“. Das Abo ist trotzdem sinnvoll, falls man die Familie verlässt oder den Familienkalender ausblendet.
5. Ferien eines anderen Bundeslandes sind eine andere Quelle und erscheinen zusätzlich, beschriftet mit dem Kürzel.

Externe Kalender einblenden

Per EventKit liest die App die Kalender des Geräts (iCloud, Google, Exchange, CalDAV) und zeigt sie grau in der eigenen Ansicht. Die Daten bleiben auf dem Gerät und werden nicht mit der Familie geteilt. Nötig ist der volle Kalenderzugriff.

Import und Migration

Weg	Ablauf	Hinweis
Gerätekalender (EventKit)	Kalender wählen, Ziel wählen, importieren	Für alte Termine muss in iOS „Alle Ereignisse“ synchronisiert werden
Geheime iCal-Adresse	Google-Adresse einfügen, einmalig importieren	Gleicher Code wie bei Abos; danach Adresse bei Google zurücksetzen
.ics-Datei	Datei mit der App öffnen, z. B. aus dem Google-Export	Google-Export ist eine ZIP-Datei, die man in der Dateien-App entpackt

Regeln für jeden Import

- Ziel wählbar: Familienkalender oder eigener Kalender.
- Duplikate über die `UID` erkennen.
- Serien samt Ausnahmen übernehmen, Zeitzonen und Ganztagstermine sauber umrechnen.
- Was verloren geht (Gäste, Anhänge, Videokonferenz-Links), vor dem Import anzeigen.

Benachrichtigungen, Widgets und Apple Intelligence

Alle drei laufen ohne Server: Erinnerungen lokal, Widgets aus der App Group, KI auf dem Gerät.

Erinnerungen

- Lokale Notifications über `UNUserNotificationCenter`. Erinnert werden die erwachsenen Teilnehmer und Teenager mit Account; ohne Teilnehmer oder nur mit Kinderprofilen alle Erwachsenen.
- iOS hält höchstens 64 geplante lokale Benachrichtigungen. Die App plant nach jedem Sync die nächsten 64 neu. Die Zeit ist eine Vorgabe am Termin bzw. Todo; jeder kann sie für sich ändern oder stummschalten (AlertPreference). Die Vorgabe beim Anlegen stellt jeder selbst ein, ab Werk 30 Minuten vorher, ganztägig am Vortag um 18 Uhr, Todos am Fälligkeitstag um 9 Uhr.
- Änderungen anderer (neuer Kommentar, neuer Artikel) kommen per CloudKit-Silent-Push. iOS drosselt diese, sie sind deshalb nur ein Extra. Sichtbare Push-Nachrichten gibt es in V1 nicht, Neues zeigt die App mit „Neu“-Markern und Badges.

Widgets

- Der Core-Data-Store liegt in einer App Group. Das Widget liest nur und synchronisiert nicht selbst mit CloudKit.
- Nach jedem Sync ruft die App `WidgetCenter.shared.reloadAllTimelines()` auf.
- Varianten: „Heute & Morgen“ (mittel), „Nächste Termine“ (klein und Sperrbildschirm).
- Optional: Todos direkt im Widget abhaken über einen App Intent.

Screenshot zu Termin

1. Nutzer teilt einen Screenshot an die App, eine Share Extension nimmt ihn an.
2. Die App prüft, ob das On-Device-Modell verfügbar ist.
3. Ab iOS 27: Bild direkt an Foundation Models, Ausgabe per `@Generable` als Terminentwurf mit Titel, Beginn, Ende, Ganztag und Ort.

4. Fallback: Texterkennung mit Vision, dann Text-Modell oder `NSDataDetector`.
5. Vorschau mit allen erkannten Terminen, Ziel-Kalender wählen, bestätigen.

Das Bild verlässt das Gerät nicht. Private Cloud Compute wird bewusst nicht genutzt.

Export und Backup

Vier Wege: ICS für Kalender, Schreiben in einen Gerätekalender (auch CalDAV), CSV für Listen und ein vollständiges JSON-Backup, das sich wiederherstellen lässt.

Format	Inhalt	Wofür
ICS	Termine eines oder mehrerer Kalender, inklusive Serien und Ausnahmen	Umzug in jede andere Kalender-App
Gerätekalender (EventKit)	Termine in einen iOS-Kalender kopieren, auch in ein CalDAV-Konto	CalDAV-Export ohne eigenen CalDAV-Client
CSV	Einkaufsliste, Todos und Termine als Tabelle	Numbers, Excel, Weiterverarbeitung
JSON-Backup	Alles: Familien, Mitglieder, Kalender, Termine, Teilnehmer, Kommentare, Listen, Abos, Farben	Vollständige Wiederherstellung in der App

- **CalDAV** ist ein Protokoll, kein Dateiformat. Ein direkter Upload auf einen CalDAV-Server bräuchte einen eigenen Client; der Weg über EventKit erreicht dasselbe, wenn das CalDAV-Konto in iOS eingerichtet ist.
- **ICS** kennt keine Kommentare oder Kinderprofile. Teilnehmer und Kommentare landen deshalb als Text in der Beschreibung.
- **Automatisches Backup:** bei allen Mitgliedern ab Werk an, wöchentlich ein JSON-Backup in den iCloud-Drive-Ordner der App, die letzten fünf werden behalten.
- **Wiederherstellen:** JSON-Datei öffnen, Objekte werden über ihre id erkannt, damit nichts doppelt angelegt wird. Ist die Familie nicht mehr erreichbar, entsteht eine neue Familie mit dem Wiederherstellenden als Owner.
- **Export starten:** über das Teilen-Menü bzw. den Datei-Dialog, pro Familie oder für alles.

Risiken, offene Fragen und Roadmap

Das größte Risiko: Die Familiendaten liegen in der iCloud des Owners.

Risiko	Auswirkung	Gegenmaßnahme
Daten in der iCloud des Owners	Löscht der Owner die Daten oder seinen Account, sind die Familiendaten weg; sie belegen seinen iCloud-Speicher	Owner-Rolle übertragbar machen (V1.1); automatisches JSON-Backup bei allen Mitgliedern ab Werk an; Wiederherstellen als neue Familie
Schema nach Release nur erweiterbar	Fehler im Modell bleiben dauerhaft	Modell vor dem ersten Release festzurren
Serientermine	Fehler bei Ausnahmen und beim Import	Früh mit echten Google-Exporten testen
Silent Pushes gedrosselt	Änderungen anderer erscheinen verzögert	Sync beim Öffnen, Erinnerungen unabhängig davon
Teilnehmer-Entitlement ab iOS 26	Ohne Entitlement keine Namen aus dem CKShare	Beantragen, zusätzlich eigenes Member-Profil
Ferien-Datenquelle	OpenHolidays ohne Uptime-Garantie; ODbL verlangt Attribution und Share-Alike	Eigener Spiegel per GitHub Action, Datenstand in der App, Attribution in der App
KI nicht auf jedem Gerät	Screenshot-Feature fehlt auf älteren Geräten	Fallback über Vision und Data Detector

Entscheidungen

- Nutzer können in mehreren Familien sein, und es lassen sich mehrere geteilte Kalender anlegen.
- Abos gelten pro Kalender; beim geteilten Kalender legt sie der Owner fest, gleiche Quellen werden nur einmal angezeigt.
- Farben der Kalender sind für jeden Nutzer änderbar.
- Kinder ohne Apple Account sind Profile; pro Termin wird gewählt, wer dabei ist.
- Veröffentlichung kostenlos im App Store, ohne In-App-Käufe.
- Name **yafo**, Mindestversion iOS/iPadOS 27, Mac als iPad-App, weltweit auf Deutsch und Englisch, Quellcode öffentlich, privater Developer Account.

- Feiertage und Ferien für DE, AT und CH aus OpenHolidays (ODbL), wöchentlich gespiegelt.
- Erinnerungen an die erwachsenen Teilnehmer, ohne Teilnehmer oder nur mit Kindern an alle Erwachsenen; Zeit als Vorgabe, pro Person änderbar.
- Keine Push-Nachrichten für Änderungen anderer in V1, nur „Neu“-Marker.
- Einkaufsliste und Todos gehören zur Familie, dazu private Listen; Todos mit zuständiger Person und Wiederholung, die beim Abhaken weiterrollt.
- Alle Erwachsenen haben Schreibrechte und dürfen alle Termine bearbeiten; Serien: nur diesen, diesen und folgende, alle.
- Kommentare lassen sich bearbeiten und löschen.
- Keine Wochenansicht in V1.
- Owner-Übertragung in V1.1; automatisches Backup bei allen ab Werk an.

Offene Fragen

- [ ] Bundle-ID `de.yafo.app` und Container `iCloud.de.yafo.app` im Developer-Portal registrieren; eine Domain ist dafür nicht nötig. Ist die ID belegt, Alternative wählen.

Roadmap

1. **Fundament:** Datenmodell, Core Data mit CloudKit, Familien anlegen und einladen, Kinderprofile, mehrere Familien, Prototyp für die Owner-Übertragung.
2. **Kalender:** Monats- und Jahresansicht, Termine mit Teilnehmern, Serien, Farbsystem, Kommentare.
3. **Listen:** Einkaufsliste und Todos.
4. **Erinnerungen und Widgets.**
5. **Abos und Import:** Feiertage, Ferien, ICS, EventKit, Google-Migration.
6. **Export und Backup:** ICS, CSV, EventKit, JSON mit Wiederherstellung. Die Owner-Übertragung folgt in V1.1 und nutzt denselben Kopiermechanismus.
7. **Screenshot zu Termin.**
8. **TestFlight** mit der eigenen Familie, danach App Store.

Quellen

- SwiftData unterstützt kein CloudKit-Sharing (Apple Developer Forums)
- SwiftData und ein zweiter CloudKit-Container (Apple Developer Forums)
- CKShare-Änderungen in iOS 26 (Apple Developer Forums)
- What’s new in the Foundation Models framework (WWDC 2026)

- OpenHolidays API und Rohdaten unter ODbL